

Adaptive Edge–Cloud Anomaly Detection Framework: A Resource-Aware Comparison of Centralized and Edge-Based Processing

1. Author information

Author: [AUTHOR FULL NAME — TO BE COMPLETED] **Affiliation:** [INSTITUTION / DEPARTMENT — TO BE COMPLETED] **Corresponding author:** [EMAIL — TO BE COMPLETED] **Challenge:** National Research & Innovation Challenge Season 3, Cloud Engineering Track **GitHub:** [Raghu2424](#) **Manuscript status:** Documentation draft based on the project summary supplied by the participant; not yet independently verified against implementation artifacts.

2. Abstract

This study compares three described anomaly-detection deployment architectures: centralized cloud processing with an LSTM autoencoder, static edge processing with an autoencoder, and a proposed resource-aware adaptive edge framework that can select among Isolation Forest, autoencoder, and LSTM-autoencoder model classes. The intended evaluation covers detection quality, latency, CPU and memory utilization, and transmitted bytes. The available project summary reports that static and adaptive edge configurations selected or used the standard autoencoder and therefore produced identical reported detection measures. Compared with the centralized configuration, both edge configurations are reported to transmit fewer bytes; the supplied data imply a 90.74% reduction. The reported latency values are 101.13 ms for centralized cloud, 0.39 ms for static edge, and 0.50 ms for adaptive edge. These apparent latency differences are not directly interpretable as a controlled end-to-end comparison because the centralized figure reportedly includes uplink, downlink, and inference, whereas the edge values represent local inference. Detection results are mixed: the centralized model has reported recall of 1.0000 but precision of 0.0797, while both edge approaches have reported accuracy of 0.8683 and F1-score of 0.1802 but recall of only 0.2128. All project measurements are reported, not independently reproduced. The present contribution is therefore a transparent comparison framework and provisional account of supplied results, not evidence that adaptive selection improves detection. Validation requires the source data, code, configuration, confusion matrices, consistent timing boundaries, repeated trials, and preferably real-world workloads.

3. Keywords

Edge computing; cloud computing; anomaly detection; edge–cloud processing; adaptive model selection; Isolation Forest; autoencoder; LSTM autoencoder; latency; communication overhead; resource-aware computing.

4. Introduction

Anomaly detection at the network edge presents a deployment trade-off: centralized processing can use cloud resources but requires moving observations across the network, whereas local inference can reduce data transfer and avoid a cloud round trip while placing computation on resource-constrained devices. Edge-computing research describes reduced response time, bandwidth cost, and data movement as motivations for processing closer to data sources, while also recognizing the limits of edge resources [2]. Mobile-edge surveys similarly describe the placement of computation near devices as a way to address latency-sensitive applications, while emphasizing joint communication and computation resource management [3].

This paper documents a reported comparison of three architectures: (A) centralized cloud detection using an LSTM autoencoder, (B) static edge detection using a standard autoencoder, and (C) a resource-aware adaptive edge approach selecting among light, standard, and heavy model classes. The comparison is intended to evaluate detection metrics alongside latency, resource use, and transmitted bytes. The project's reported summary is the sole source for its experimental values; no source code, input dataset, machine-readable original results, or execution environment was available for independent inspection in this preparation.

Accordingly, the contribution claimed here is deliberately bounded: a documented architecture comparison and transparent transcription and interpretation of the supplied results, with methodological requirements for a reproducible evaluation. The report does not claim that the adaptive system outperforms the static edge baseline in detection: the reported detection metrics are identical.

5. Background and motivation

5.1 Edge–cloud processing

Edge computing moves some processing from centralized infrastructure closer to data generation, motivated by response-time, bandwidth, energy, and data-safety considerations [2]. The resulting design space is not a simple choice between “cloud” and “edge”: edge-intelligence literature describes device, edge, and cloud execution modes, and frames latency, accuracy, memory, communication, and energy as competing performance dimensions [4].

5.2 Anomaly-detection models

Isolation Forest (iForest) identifies unusual samples using isolation rather than distance or density calculations; the original method is described as supporting low linear time complexity and low memory use, and as applicable when anomalies are not available in training data [5]. These properties make it a plausible light candidate in the supplied architecture, but this manuscript has not verified the project’s implementation or comparative resource cost.

A published LSTM encoder-decoder anomaly-detection method learns normal time-series behavior through reconstruction and uses reconstruction error for anomaly detection; its authors report evaluations on several public and real-world time-series collections [6]. A separate spacecraft-telemetry study combines LSTM modeling with nonparametric dynamic thresholding and discusses false-positive mitigation [7]. Those publications motivate sequence-model approaches; they do not verify the project’s implementation, model quality, or reported numbers. The project describes its autoencoder and LSTM-autoencoder as STANDARD and HEAVY model categories respectively; exact layers, training, scoring and threshold procedures remain [TO BE VERIFIED].

5.3 Resource-aware model selection

Prior embedded-systems research has proposed adaptive selection among pretrained models based on accuracy and inference-time constraints, with experiments on image classification and an embedded platform [8]. Edge-intelligence research further emphasizes that selecting a model for edge execution must account for runtime performance and resource consumption rather than predictive quality alone [4]. These precedents establish that the general design problem has prior work; this study therefore makes no claim to invent adaptive edge model selection. The project-specific question is whether the stated three-way architecture comparison, utility-based adaptation, and communication accounting yield a reproducible advantage under one consistent evaluation protocol.

6. Problem statement

The project addresses a multi-objective deployment problem: how to locate anomaly-detection inference among cloud and edge resources, and how to select among models of different described resource profiles, while balancing detection quality, latency, CPU and memory utilization, and network traffic. The supplied results do not yet establish that the three architectures were measured at equivalent boundaries or under repeated and independently auditable conditions. In particular, the reported centralized latency includes network transfer and inference while the edge latency represents local inference; this boundary mismatch prevents treating the reported latency reduction as a like-for-like end-to-end effect.

7. Research objectives

1. Compare the three described processing architectures on anomaly-detection metrics.
2. Compare reported latency, CPU utilization, memory utilization, and transmitted bytes.
3. Document how resource-aware model and processing decisions are intended to be made.

4. Assess whether adaptation offers a measurable benefit over the fixed standard-model edge baseline, without presuming that it does.
5. Specify the artifacts and consistent measurement protocol required to reproduce and validate the comparison.

8. Research questions

- **RQ1:** How do centralized cloud, static edge, and adaptive edge processing compare in reported detection accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC?
- **RQ2:** How do the three architectures compare in latency, CPU utilization, memory utilization, and bytes transmitted under a consistent, end-to-end measurement definition?
- **RQ3:** Does resource-aware adaptation improve the joint quality–resource trade-off relative to static edge processing, and under which resource states and decision-policy settings?
- **RQ4:** Do the reported outcomes persist across repeated runs, alternative thresholds, and validation on real-world data?

9. Research hypotheses (to be tested, not established)

- **H1:** Edge-based approaches will transmit fewer bytes than the centralized-cloud baseline when only anomaly events are sent upstream.
- **H2:** Under a common end-to-end latency boundary and controlled network conditions, local or edge inference will reduce latency relative to centralized processing for at least some workload and network settings.
- **H3:** The adaptive architecture will improve a prespecified joint utility of detection quality and resource cost relative to static edge processing, without violating a pre-defined minimum-recall or minimum-F1 constraint.
- **H4:** The adaptive architecture will be non-inferior to static edge detection on a pre-specified primary detection measure within a margin justified before testing.

H1–H4 are proposed hypotheses. The values in this draft do not establish them. In particular, identical reported detection metrics for static and adaptive edge do not support a claim of improved adaptive detection.

10. Related work and literature review

Edge-computing foundations motivate near-source processing through response-time and bandwidth considerations [2]. Mobile-edge computing research treats offloading and resource allocation as joint communication–computation decisions [3]. Edge-intelligence literature reviews centralized, device, edge, and edge–cloud inference arrangements

and identifies resource-aware model design and runtime trade-offs as open engineering concerns [4].

On the detection side, Isolation Forest provides an isolation-based approach designed to avoid distance/density calculations and to limit computational and memory requirements [5]. LSTM encoder–decoder methods model normal sequences by reconstruction, with anomalies identified from reconstruction error [6]. LSTM telemetry research also shows that thresholding and false-positive mitigation are integral to practical anomaly detection, not merely details secondary to model choice [7].

Resource-aware selection is not new as a general concept. Taylor and colleagues describe choosing among pretrained models according to accuracy and inference-time constraints on embedded systems [8]. The present project's reported framing differs in its proposed juxtaposition of centralized detection, static edge detection, and utility-driven edge adaptation, while counting both network transmission and host resource indicators. Whether this particular evaluation adds a robust empirical result remains unresolved until its implementation and measurement protocol are available.

11. Research gap

The literature retrieved for this draft documents edge/cloud placement, edge-intelligence resource trade-offs, anomaly detection methods, and adaptive model selection as established topics [2], [4], [5], [8]. It does not establish that no prior study has compared these approaches. Within the retrieved passages, no directly comparable primary evaluation was identified that simultaneously matches this project's specific three-arm combination (centralized LSTM autoencoder, fixed standard edge autoencoder, and resource-aware model selection), event-only edge communication, and the stated bundle of detection and system measures under a common timing boundary. This is a bounded gap in the retrieved evidence, not a claim that such work does not exist.

A defensible contribution is thus a reproducible, controlled evaluation of that specific comparison, if the implementation and experimental records substantiate it. The immediate gap in the current report is evidentiary: the project summary supplies outcomes but not the files needed to check the dataset, labels, confusion matrices, thresholds, utility function, hardware/network context, timing boundaries, or run-to-run variability.

12. Proposed architecture

The following is a documentation-level representation of the supplied design description, not a code inspection. The centralized baseline sends all observations to the cloud and uses the heavy LSTM autoencoder. The static baseline runs the standard autoencoder locally and reports only anomaly events. The adaptive approach uses an AdaptiveController to make resource-aware decisions and an EdgeProcessor that can select light, standard, or heavy models; a CloudCoordinator represents centralized processing, and MetricsCalculator/SystemMonitor represent evaluation and monitoring

components. Names and roles are supplied descriptions and must be checked against source files.

13. Detailed system design

13.1 Components

- **Adaptive controller** (`src/adaptive/AdaptiveController`): described as applying a resource-aware utility decision mechanism. The objective function, feature inputs, thresholds, weights, decision cadence, and fallback rules are [TO BE VERIFIED].
- **Edge processor** (`src/edge/EdgeProcessor`): described as performing local inference and model selection. Its use of the light Isolation Forest, standard autoencoder, and heavy LSTM autoencoder is a project-summary description, not implementation-verified.
- **Cloud coordinator** (`src/cloud/CloudCoordinator`): described as the central processing route used by the centralized baseline. Whether it also receives edge events, model updates, or only centralized input is [TO BE VERIFIED].
- **Metrics calculator and system monitor** (`src/evaluation/`): described as calculating detection results and monitoring system performance. Metric definitions, sampling frequency, units, and instrumentation are [TO BE VERIFIED].

13.2 Processing and communication flow

For centralized processing, all input data is transmitted to the cloud for inference using the LSTM-autoencoder model. For static edge processing, the fixed autoencoder processes input locally and only anomaly events are sent. For adaptive processing, resource state and a utility-based policy are described as determining local processing/model choice or transmission; the supplied summary says the standard model was primarily selected under reported resource constraints, and the reported communication figure is event-only. The conditions for bypassing local inference or sending raw observations require implementation verification.

13.3 Resource-awareness considerations

A resource-aware policy should define its measured resource state (for example CPU/memory availability), decision interval, model cost estimates, quality/latency objective, and constraints. It should also record every selected model and action, with the contemporaneous resource state and resulting decision. These are design and reproducibility recommendations, not facts about the available project implementation. The reported utility weights may need target-specific calibration, according to the supplied methodology notes.

13.4 Known design limitations

The supplied summary reports that the adaptive method primarily selected the standard model, which would explain why its reported detection measures equal those of static edge processing. The reported adaptive run also has 0.50 ms latency versus 0.39 ms for static edge, with higher CPU and memory utilization. The current report does not show the controller's decision trace, transitions among models, or a comparison under resource states where adaptation changes the selected action.

14. Methodology

This manuscript records a three-architecture comparative evaluation as described by the participant. The project summary states that the experiment used synthetic data; threshold calibration reportedly used validation data and optimized F1 to avoid test-set leakage; fixed random seeds were reportedly configured in `default_config.yaml`; Python 3.14.6 compatibility was reported; and the adaptive approach primarily selected the standard model under current constraints. Each item is a supplied methodological description and remains [TO BE VERIFIED] against the repository and raw experiment outputs.

A reproducible verification should document a single held-out test set, training/validation/test partition boundaries, any temporal or entity grouping, label-generation procedure, preprocessing, threshold-selection rule, software and dependency versions, random seeds, model configurations, hardware and network conditions, and exact run commands. Repeated independent runs should report dispersion and uncertainty. The study should select its primary endpoint(s) in advance and avoid tuning decisions on the final test set.

15. Experimental setup

The supplied summary identifies synthetic data but does not provide the generator, data volume, class prevalence, anomaly types, temporal structure, labels, or split sizes. No hardware specification, network conditions, software environment record, training epochs, model hyperparameters, or number of repeated runs was supplied. These fields must remain unspecified until evidence is provided.

The reported system metrics list centralized latency as a total including uplink, downlink, and inference, and edge latency as local inference. Those are different measurement boundaries. A valid comparison should report both component and end-to-end latency consistently for each architecture, using the same start/end events, workload, device/network conditions, warm-up policy, and aggregation statistic. In addition, report bytes sent in each direction, define whether protocol overhead and event metadata are counted, and separate per-event from total-run traffic.

16. Evaluation metrics and definitions

The following are conventional interpretations of the metric names; confirm the exact implementation, averaging convention, and thresholding before publication.

- **Accuracy:** proportion of all evaluated examples classified correctly.
- **Precision:** proportion of predicted positive cases that are true positives.
- **Recall (sensitivity):** proportion of actual positive cases detected.
- **F1-score:** harmonic mean of precision and recall at a selected threshold.
- **ROC-AUC:** area under the receiver operating characteristic curve across score thresholds.
- **PR-AUC:** area under the precision–recall curve across score thresholds; its implementation and interpolation convention should be stated.
- **Latency:** elapsed processing time under a clearly specified boundary; report end-to-end and component timings separately.
- **CPU utilization / memory utilization:** host resource measures. Sampling interval, process/system scope, peak/mean aggregation, and monitoring overhead need to be specified.
- **Data transmitted:** number of bytes sent over the network during a defined evaluation interval, with directions, protocol overhead, and per-example/total basis defined.

For anomaly data, accuracy alone can be misleading when normal and anomalous cases are imbalanced. Report class counts and prevalence, confusion matrices, precision/recall and PR-AUC, and threshold behavior alongside ROC-AUC. The supplied metrics are insufficient to reconstruct confusion matrices because sample counts are absent.

17. Reported results

All values in Tables 1–2 are reported results supplied in the project summary, described there as sourced from `results/final_results.json` and validated on 2026-09-09. They have not been independently reproduced in this conversation or checked against the named JSON artifact.

Table 1. Reported detection metrics

Metric	Centralized Cloud	Static Edge	Adaptive Edge
Accuracy	0.2142	0.8683	0.8683
Precision	0.0797	0.1562	0.1562
Recall	1.0000	0.2128	0.2128
F1-score	0.1476	0.1802	0.1802
ROC-AUC	0.6190	0.6128	0.6128
PR-AUC	0.0814	0.1123	0.1123

Table 2. Reported system-performance metrics

System metric	Centralized Cloud	Static Edge	Adaptive Edge
Latency	101.13 ms	0.39 ms	0.50 ms
CPU utilization	59.66%	37.90%	47.61%
Memory utilization	77.88%	77.30%	79.90%
Data transmitted	331,680 bytes	30,720 bytes	30,720 bytes

Table 3. Derived arithmetic from the supplied values

Comparison	Derived value	Basis and caveat
Static Edge vs Centralized latency reduction	99.61%	Arithmetic from 101.13 ms and 0.39 ms; not a like-for-like end-to-end comparison because reported timing boundaries differ.
Adaptive Edge vs Centralized latency reduction	99.51%	Arithmetic from 101.13 ms and 0.50 ms; same boundary caveat.
Edge data-transmission reduction vs Centralized	90.74%	Arithmetic from 331,680 bytes and 30,720 bytes for either edge approach.
Adaptive latency vs Static Edge	0.11 ms higher (28.21% relative)	Arithmetic from 0.50 ms and 0.39 ms; supplied summaries do not define timing variability.

The first three derived percentages are calculated from the supplied values, not independent measurements. The exact unrounded arithmetic is preserved in results/reported_results_transcribed.json.

18. Results interpretation

The centralized configuration's reported recall of 1.0000 and precision of 0.0797 indicate that it detected all positive examples in the reported evaluation while also generating many false-positive predictions, assuming the stated metrics use the conventional positive-class definitions. Its reported accuracy is 0.2142 and F1-score is 0.1476. Static and adaptive edge have identical reported detection metrics: accuracy 0.8683, precision 0.1562, recall 0.2128, F1 0.1802, ROC-AUC 0.6128, and PR-AUC 0.1123. Their higher reported accuracy and F1 do not imply uniformly better anomaly detection: recall of 0.2128 means that many actual positive cases may have been missed. Without the class distribution, confusion matrices, threshold, and denominator counts, the operational magnitude cannot be determined.

Both edge configurations have identical reported transmitted bytes, 30,720 versus 331,680 for centralized processing. The arithmetic difference is approximately 90.74%. This is consistent with the described strategy of sending anomaly events rather than all observations, but event-only semantics and counted network layers need verification. The reported adaptive configuration consumes more CPU and memory than the static edge configuration and records 0.50 ms rather than 0.39 ms latency. Since its reported detection metrics are identical, these results do not demonstrate a detection-quality gain from adaptation.

The latency ratios should not be presented as evidence of a measured end-to-end speedup: the centralized and edge timing boundaries differ according to the supplied caveat. A same-boundary, end-to-end comparison could yield a different result.

19. Discussion

The reported results illustrate a potentially meaningful communication trade-off, but the strength of that conclusion depends on whether data accounting is complete and the three systems process the same input volume. The identical byte totals for static and adaptive edge are compatible with the described event-only output pattern; by themselves they do not show that the adaptive controller made better decisions.

The detection comparison is a trade-off, not a clear winner. Centralized cloud reports perfect recall but very low precision; the two edge configurations report better accuracy and F1 while sacrificing recall substantially. A deployment decision should therefore depend on the relative costs of missed anomalies and false alarms, the target prevalence, and thresholds chosen for the intended use. No universal superiority follows from the six aggregate metrics.

A more informative next evaluation would include: (i) confusion matrices and class prevalence; (ii) threshold-independent score curves and fixed-threshold operating points; (iii) matched end-to-end timing, with component timings; (iv) bytes per observation and per alert; (v) run-level distributions across repeated trials; and (vi) an ablation showing when the adaptive controller switches between light, standard, and heavy models. Such testing would turn the current provisional comparison into an empirical assessment of whether utility-based adaptation changes system outcomes.

20. Threats to validity

- **Construct validity:** “latency,” “CPU utilization,” “memory utilization,” and “data transmitted” lack full operational definitions. Centralized and edge latency use different reported boundaries.
- **Internal validity:** It is unknown whether architectures use identical splits, labels, preprocessing, model-selection opportunities, or threshold-selection rules. Fixed seeds and validation-only F1 tuning are reported, not verified.
- **Statistical conclusion validity:** The number of runs, uncertainty intervals, variance, statistical tests, and run independence are absent. The supplied results cannot establish statistical significance.
- **External validity:** The experiment reportedly uses synthetic data. Generalization to real traffic, sensor telemetry, device populations, anomaly types, and network environments is unknown.
- **Measurement validity:** Instrumentation overhead, hardware, network conditions, warm-up, timing clock, transfer protocol, and byte-accounting boundary have not been provided.
- **Adaptive-policy validity:** The adaptive approach reportedly selects STANDARD primarily; results may not exercise the full policy or represent conditions where switching is valuable.

21. Limitations

The project data and results are reported by the participant and have not been independently reproduced. The code and configuration were not inspected, and `results/final_results.json` was not available in this conversation. The reported synthetic dataset, Python version, seeds, threshold calibration and component descriptions remain unverified. No sample size, class balance, dataset generator, hardware, network profile, hyperparameters, confidence intervals, or repeated-run statistics can be supplied. The literature discussion is bounded by the academic sources retrieved under the selected source restrictions and abstracts; full texts were not fetched. No official repository instructions could be checked because web access was not enabled.

22. Ethical and reproducibility considerations

Do not describe the supplied results as independently validated, published, statistically significant, or code-verified. Keep a traceable distinction among participant-reported results, arithmetic derived from those values, literature-derived statements, and proposed future tests. Before public submission, include the data-generation script or a precise data specification, configuration files, environment/dependency lock, source revision identifier, evaluation scripts, raw outputs, seeds, model artifacts or deterministic training instructions, and a README with commands only after confirming they work.

If data or telemetry could contain sensitive information, document provenance, consent/authorization, minimization, and de-identification; no claim about the current dataset's sensitivity can be made from the supplied description.

23. Future work

1. Verify the architecture, decision function, model definitions, source paths, metric calculations, thresholds, and system-monitoring methods against the actual project files.
2. Recover and publish the synthetic data generator or dataset, its class prevalence and anomaly-generation rules, partitions, and event labels.
3. Rerun all three approaches with matched data, matched threshold procedure, consistent timing boundaries, and repeated trials.
4. Report confusion matrices, class supports, per-class metrics, confidence intervals or run-level variability, and threshold sensitivity.
5. Measure end-to-end latency for each method, separately reporting uplink, queueing, inference, downlink, and event-publication stages where applicable.
6. Log resource state, selected model, controller decision, utility components, decision timing, and bytes transmitted for each run.
7. Compare on real-world data and under controlled network and resource scenarios; report results separately rather than pooling non-comparable settings.
8. Predefine acceptance constraints for missed anomalies, false alarms, resource use, and latency, and evaluate the adaptive policy against those targets.

24. Conclusion

This documentation compares three supplied edge–cloud anomaly-detection designs and transcribes the reported results without treating them as independently verified. The reported edge configurations send fewer bytes than centralized cloud, while the detection measures show a recall-versus-precision trade-off and no reported detection difference between static and adaptive edge. The apparent latency reductions are not directly comparable because the timing boundaries reportedly differ. The adaptive system's reported results therefore do not establish an improvement over static edge. The defensible next step is to verify the source artifacts and rerun a controlled, repeated evaluation with consistent end-to-end timing, class-aware reporting, and explicit adaptation traces.

25. References

The verified publication metadata and stable DOI identifiers for the literature cited above are catalogued in `references/literature_review.md`. That bibliography distinguishes peer-reviewed publications from preprints and records entries for which author/venue

metadata remain [TO BE VERIFIED]. Complete the final required citation style and author details before submission. The contribution text above cites sources using the document's exact evidence markers; those markers should be replaced by stable repository-style citation links or a finalized academic reference format if the submission platform does not render them.

Additional references

- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, 2016, doi: [10.1109/JIOT.2016.2579198](https://doi.org/10.1109/JIOT.2016.2579198).
- [3] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, 2017, doi: [10.1109/comst.2017.2745201](https://doi.org/10.1109/comst.2017.2745201).
- [4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, June 2019, doi: [10.1109/jproc.2019.2918951](https://doi.org/10.1109/jproc.2019.2918951).
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Transactions on Knowledge Discovery from Data*, 2012, doi: [10.1145/2133360.2133363](https://doi.org/10.1145/2133360.2133363).
- [6] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," *arXiv*, 2016, Available: <https://arxiv.org/pdf/1607.00148v2>
- [7] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," 2018, doi: [10.1145/3219819.3219845](https://doi.org/10.1145/3219819.3219845).
- [8] B. Taylor, V. S. Marco, W. Wolff, Y. Elkhatib, and Z. Wang, "Adaptive deep learning model selection on embedded systems," *Lancaster EPrints (Lancaster University)*, pp. 31–43, 2018, doi: [10.1145/3211332.3211336](https://doi.org/10.1145/3211332.3211336).